

Data Structure & Algorithms

Rapid-Revision One-Liners

IBPS SO (IT OFFICER)

COMPLETE PREPARATION SERIES

100% SYLLABUS COVERAGE

CONCEPT-FOCUSED LEARNING

MCQ-ORIENTED APPROACH

QUICK REVISION READY

COMPILED BY

Asabhya Maanav

2026 EDITION

VERSION 1.0

Table of Contents

Data Structure & Algorithms — One-Liners	3
Introduction, ADT & Algorithm Analysis	3
Arrays & Matrices	3
Linked Lists	3
Stacks	4
Queues	4
Trees	4
Heaps & Heapsort	5
Graphs	5
Searching	6
Sorting Fundamentals	6
Sorting Algorithms	6
Merging	6
Hashing	7
PASCAL & Programming Constructs	7
Deep-Dive Drill — Complexity & Recursion	7
Deep-Dive Drill — Arrays & Matrices	7
Deep-Dive Drill — Linked Lists	8
Deep-Dive Drill — Stacks	8
Deep-Dive Drill — Queues	8
Deep-Dive Drill — Trees	8
Deep-Dive Drill — Graphs	9
Deep-Dive Drill — Sorting & Searching	9
Deep-Dive Drill — Hashing	9
Rapid-Fire Mixed Exam Pointers	10
Atomic Round 2 — Trees & Graph Theory	10
Atomic Round 2 — Searching, Sorting & Hashing	10
Atomic Round 2 — Operations, Conditions & Values	11
Final Rapid-Fire — Discriminators	11
Atomic Round 3 — Advanced Structures & Algorithms	12

Data Structure & Algorithms — One-Liners

Introduction, ADT & Algorithm Analysis

1. An algorithm has **5** characteristics: **finiteness, definiteness, input, output and effectiveness**.
2. Data structures split into **2** broad classes: **primitive** (int, char, float, pointer) and **non-primitive** (array, list, tree, graph).
3. There are **2** non-primitive arrangements: **linear** (array, stack, queue, list) and **non-linear** (tree, graph).
4. Data structures are **2** types by memory: **static** (fixed-size array) and **dynamic** (linked list).
5. The **7** core operations on data structures are **traversal, insertion, deletion, searching, sorting, merging and updating**.
6. An **ADT** specifies **2** views: the **what** (logical interface) and hides the **how** (physical implementation).
7. There are **5** major algorithm design techniques: **Divide-and-Conquer, Greedy, Dynamic Programming, Backtracking and Branch-and-Bound**.
8. Complexity is analysed in **3** cases: **best (Ω), average and worst (O)** case.
9. There are **5** asymptotic notations: **Big-O (upper), Big-Omega (lower), Big-Theta (tight), little-o and little-omega**.
10. **Big-O** gives the **upper bound** while **Big-Omega (Ω)** gives the **lower bound** of running time.
11. **Big-Theta (Θ)** gives the **tight bound**, valid only when upper and lower bounds **match**.
12. The standard growth ordering is **$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$** .
13. **Binary search** runs in **$O(\log n)$** because it **halves** the range each step.
14. **Constant time $O(1)$** examples are **array indexing, stack push/pop and hash lookup**.
15. **Exponential $O(2^n)$** arises in problems like **naive recursive Fibonacci and subset generation**.
16. The **Master's Theorem** form is **$T(n) = aT(n/b) + f(n)$** comparing $f(n)$ with $n^{\log_b a}$.
17. Master's Theorem has **3** cases deciding whether **leaves, balance, or the root** dominate the cost.
18. **Merge sort's** recurrence **$T(n)=2T(n/2)+n$** solves to **$\Theta(n \log n)$** by Master's Case 2.
19. There are **3** recurrence-solving methods: **substitution, recursion-tree and Master's Theorem**.
20. **Amortized analysis** has **3** methods: **aggregate, accounting (banker's) and potential**.
21. A **dynamic array's** insertion is **amortized $O(1)$** despite occasional **$O(n)$** resizing.
22. The **time-space trade-off** is shown by **hashing**, which spends **extra space** for **$O(1)$** time.

Arrays & Matrices

23. A **1-D array** address is **$B + (i - LB) \times w$** , where **w** is element size in bytes.
24. Arrays support **2** storage orders: **row-major** (C/C++) and **column-major** (FORTRAN/MATLAB).
25. **Row-major** address multiplies the row index by the **number of columns**; **column-major** by the **number of rows**.
26. Arrays give **$O(1)$** random access but **$O(n)$** insertion/deletion due to **shifting**.
27. A **lower/upper triangular** matrix stores **$n(n+1)/2$** non-zero elements.
28. A **tri-diagonal** matrix has only **$3n - 2$** non-zero elements (main + 2 adjacent diagonals).
29. A **sparse matrix** is best stored as a **triplet (row, column, value)** whose first row holds **(rows, cols, count)**.
30. A **diagonal matrix** needs only **n** stored elements (the main diagonal).
31. **Fast transpose** of a sparse matrix runs in **$O(\text{cols} + \text{non-zeros})$** time.
32. The **2** advantages of arrays are **$O(1)$ random access** and **cache locality**; the main limitation is **fixed size**.

Linked Lists

33. A linked-list node has **2** parts in an SLL: **DATA and NEXT (pointer)**.
34. A **doubly linked list (DLL)** node has **3** parts: **PREV, DATA and NEXT**.
35. There are **4** linked-list types: **singly, doubly, circular and header** linked lists.
36. In a **circular linked list** the last node's link points to the **first node**, not **NULL**.
37. Insertion/deletion at the **beginning** of a linked list is **$O(1)$** ; at the **end** it is **$O(n)$** ($O(1)$ with a tail pointer).
38. **Searching** in a linked list is **$O(n)$** because access is **sequential**, not random.
39. **Floyd's cycle detection** uses **2** pointers: **slow (1 step)** and **fast (2 steps)** — the "tortoise and hare".

40. Iterative reversal of a linked list uses **3** pointers — **prev, curr, next** — in **$O(n)$** time and **$O(1)$** space.
41. The **middle** of a linked list is found in **1 pass** using the **slow/fast pointer** technique.
42. Linked lists have **4** key applications: **polynomial arithmetic, sparse matrices, dynamic memory and adjacency lists**.
43. In **polynomial representation** each node stores **2** fields: **coefficient and exponent**.
44. Compared to arrays, linked lists trade **$O(1)$ insertion** for **$O(n)$ access** and add **pointer** overhead.
45. A **header linked list** keeps a special **header node** storing global info like the **node count**.

Stacks

46. A **stack** follows the **LIFO (Last-In-First-Out)** principle with all operations at the **TOP**.
47. The **5** stack operations are **push, pop, peek/top, isEmpty and isFull**.
48. Stack **overflow** occurs when **$TOP == size - 1$** (push on full); **underflow** when **$TOP == -1$** (pop on empty).
49. Stack push and pop are both **$O(1)$** in array and linked implementations.
50. There are **3** expression notations: **infix (A+B), prefix/Polish (+AB) and postfix/Reverse-Polish (AB+)**.
51. In **prefix** the operator comes **before** operands; in **postfix** it comes **after** operands.
52. **Postfix evaluation** uses **1** stack: push operands, and on an operator pop **2** operands.
53. Operator precedence (high→low) is **$^$ (exponent) > * / (multiply/divide) > + - (add/subtract)**.
54. The **Tower of Hanoi** with **n** disks needs exactly **$2^n - 1$** moves.
55. **Recursion** uses the **system/call stack**, storing one **activation record (stack frame)** per call.
56. **6+** stack applications include **expression conversion, parenthesis matching, recursion, string reversal, backtracking and undo**.
57. Two stacks can share **1** array by growing from **both ends** toward the middle for best space use.
58. **Infix-to-postfix** conversion pushes **(**, and on **)** pops operators **until** the matching **(**.

Queues

59. A **queue** follows the **FIFO (First-In-First-Out)** rule: insert at **REAR**, delete at **FRONT**.
60. The queue operations are **enqueue (rear), dequeue (front), front, rear, isEmpty and isFull**.
61. There are **4** queue types: **linear, circular, double-ended (deque) and priority queue**.
62. A **linear array queue** suffers **false overflow** because dequeued **front** cells cannot be reused.
63. A **circular queue** wraps using **$rear = (rear + 1) \% SIZE$** , reusing freed slots.
64. A circular queue is **full** when **$(rear + 1) \% size == front$** and **empty** when **$front == -1$** .
65. A **deque** has **2** restricted forms: **input-restricted** and **output-restricted** deque.
66. A **priority queue** serves the **highest-priority** element first and is best built on a **heap ($O(\log n)$)**.
67. Queues power **4+** applications: **CPU scheduling, BFS, buffering/spooling and interrupt handling**.
68. A **queue using 2 stacks** and a **stack using 2 queues** are classic interconversion problems.
69. **BFS** graph/tree traversal is implemented using a **queue** data structure.

Trees

70. A tree with **n** nodes has exactly **$n - 1$ edges** and a single **root**.
71. A **binary tree** node has **at most 2** children; max nodes at level **$l = 2^l$** .
72. A binary tree of height **h** has at most **$2^{h+1} - 1$** nodes and a minimum height of **$\lceil \log_2 n \rceil$** .
73. In any binary tree, leaf count **$L = n_{\blacksquare} + 1$** ($n_{\blacksquare}$ = nodes with two children).
74. A binary tree with **n** nodes has exactly **$n + 1$ NULL** (empty) child links.
75. There are **6+** binary-tree types: **full, complete, perfect, skewed, balanced and extended (2-tree)**.
76. A **full/strictly** binary tree has every node with **0 or 2** children; a **complete** tree fills the last level **left-to-right**.
77. A **perfect** binary tree has all leaves at the **same level** and **$2^{h+1} - 1$** nodes.
78. There are **3** depth-first traversals: **inorder (LNR), preorder (NLR) and postorder (LRN)**.
79. **Inorder** traversal of a **BST** yields keys in **sorted ascending** order.
80. **Preorder's first** node and **postorder's last** node both give the **root** of the tree.

81. A unique binary tree needs **Inorder + Preorder** or **Inorder + Postorder**; **inorder is mandatory**.
82. **Level-order (BFS)** traversal of a tree uses a **queue** and runs in **$O(n)$** .
83. **BST** search/insert/delete are **$O(\log n)$** on average but **$O(n)$** in the worst (skewed) case.
84. **BST deletion** has **3** cases: **leaf, one child, and two children** (replace by inorder successor/predecessor).
85. An **AVL tree** keeps **balance factor** $\in \{-1, 0, +1\}$, where **BF = height(left) – height(right)**.
86. AVL rebalancing uses **4** rotations: **LL, RR (single) and LR, RL (double)**.
87. An **LL** imbalance needs **1** right rotation; an **LR** imbalance needs **2** rotations (left then right).
88. AVL operations are guaranteed **$O(\log n)$** since its height stays about **$1.44 \log n$** .
89. A **B-tree of order m** has at most **m children** and **m – 1 keys**, with all **leaves at the same level**.
90. A non-root B-tree node has at least **$\lceil m/2 \rceil$ children** (i.e., **$\lceil m/2 \rceil - 1$ keys**).
91. A **B+ tree** stores all data in **leaf nodes only**, linked together for fast **range queries**.
92. A **Red-Black tree** obeys **5** properties and guarantees **$O(\log n)$** by balancing **black-height**.
93. A **threaded binary tree** replaces **NULL** pointers with **threads** to inorder predecessor/successor.
94. An **expression tree** has **operands as leaves** and **operators as internal nodes**.
95. An **OBST** minimises the **weighted (expected) search cost** and is built via **DP** in **$O(n^3)$** .
96. The number of distinct binary trees with **n** nodes equals the **n-th Catalan number = $C(2n, n)/(n+1)$** .
97. Catalan numbers for $n = 0..5$ are **1, 1, 2, 5, 14, 42**.

Heaps & Heapsort

98. A **heap** is a **complete binary tree** stored in an **array**, satisfying the heap property.
99. There are **2** heap types: **max-heap** (root is maximum) and **min-heap** (root is minimum).
100. Heap array indices (1-based) are **left = $2i$, right = $2i + 1$, parent = $\lceil i/2 \rceil$** .
101. Heap **insertion** and **root deletion** are both **$O(\log n)$** via **sift-up/sift-down**.
102. **Build-Heap** runs in **$O(n)$** — a tight bound, NOT **$O(n \log n)$** .
103. **Peek (getMax/getMin)** on a heap is **$O(1)$** .
104. **Heapsort** runs in **$O(n \log n)$** in **all 3 cases** and is **in-place** with **$O(1)$** space.
105. Heapsort is **unstable**, unlike merge sort which is **stable**.
106. A **priority queue** built on a heap supports **insert/extract in $O(\log n)$** and **peek in $O(1)$** .

Graphs

107. A graph **G = (V, E)** has **2** parts: a **vertex set V** and an **edge set E**.
108. The **handshaking lemma** states the **sum of all degrees = $2 \times |E|$** .
109. A **complete undirected** graph has **$n(n-1)/2$ edges**; a **complete directed** graph has **$n(n-1)$** .
110. There are **3** graph representations: **adjacency matrix, adjacency list and incidence matrix**.
111. An **adjacency matrix** uses **$O(V^2)$** space; an **adjacency list** uses **$O(V + E)$** , ideal for **sparse** graphs.
112. An adjacency matrix checks an edge in **$O(1)$** but wastes space for **sparse** graphs.
113. **BFS** uses a **queue** and **DFS** uses a **stack/recursion**, both running in **$O(V + E)$** .
114. **BFS** finds the **shortest path in an unweighted** graph; **DFS** is used for **cycle detection and topological sort**.
115. A **spanning tree** of **n** vertices has exactly **n – 1 edges** and **no cycle**.
116. There are **2** classic MST algorithms: **Prim's and Kruskal's**, both **greedy**.
117. **Prim's** runs in **$O(V^2)$** (matrix) or **$O(E \log V)$** (heap), best for **dense** graphs.
118. **Kruskal's** runs in **$O(E \log E)$** using **Union-Find**, best for **sparse** graphs.
119. A graph can have **multiple MSTs** but the **total minimum weight is unique**.
120. **Dijkstra's** finds single-source shortest paths in **$O((V+E) \log V)$** but **fails on negative** edges.
121. **Bellman-Ford** handles **negative weights** and detects **negative cycles** in **$O(V \times E)$** .
122. **Floyd-Warshall** computes **all-pairs shortest paths** via **DP** in **$O(V^3)$** .
123. **Warshall's** algorithm finds the **transitive closure** (reachability) in **$O(V^3)$** .
124. **Dijkstra and Prim** are **greedy**, whereas **Bellman-Ford and Floyd-Warshall** are **dynamic programming**.

- 125. **Topological sort** exists **only for a DAG** and runs in $O(V + E)$ via DFS or **Kahn's** algorithm.
- 126. **Union-Find** with **path compression + union by rank** gives near- $O(\alpha(n))$ (inverse Ackermann) operations.
- 127. An **articulation point/bridge** is a vertex/edge whose removal **disconnects** the graph, found by DFS in $O(V+E)$.
- 128. A digraph is **strongly connected** if there is a **directed path both ways** between every vertex pair.
- 129. **BFS** explores **level-by-level** while **DFS** explores **depth-first** along each branch.

Searching

- 130. **Linear search** runs in $O(n)$ worst case and works even on **unsorted** data.
- 131. **Binary search** runs in $O(\log n)$ but **requires a sorted** array.
- 132. The safe mid formula is $\text{mid} = \text{low} + (\text{high} - \text{low})/2$ to avoid **integer overflow**.
- 133. **Interpolation search** averages $O(\log \log n)$ on **uniformly distributed** data, degrading to $O(n)$.
- 134. **Fibonacci search** runs in $O(\log n)$ using only **addition/subtraction**, avoiding division.
- 135. A **sentinel** linear search removes the bound check by placing the key at the **end**.
- 136. The **best case** of binary, interpolation and Fibonacci search is $O(1)$ (target at the first probe).
- 137. Among searches, only **linear search** works on **unsorted** input; the rest need **sorted** data.

Sorting Fundamentals

- 138. Sorting is **2** types by location: **internal** (in RAM) and **external** (on disk/tape).
- 139. A **stable** sort preserves the relative order of **equal keys**; an **unstable** sort may not.
- 140. **5** stable sorts are **Bubble, Insertion, Merge, Counting and Radix**.
- 141. **3** unstable sorts are **Selection, Quick and Heap** (and Shell).
- 142. **In-place** sorts use $O(1)$ extra space — **Bubble, Insertion, Selection, Heap, Quick**.
- 143. **Non-comparison** sorts are **Counting, Radix and Bucket**, beating the $\Omega(n \log n)$ comparison bound.
- 144. The **lower bound** for any comparison-based sort is $\Omega(n \log n)$ (decision-tree model).

Sorting Algorithms

- 145. **Bubble sort** is $O(n^2)$ average/worst but $O(n)$ best (optimised), **stable** and **in-place**.
- 146. **Insertion sort** is $O(n^2)$ worst, $O(n)$ best, **stable**, and ideal for **nearly-sorted/small** data.
- 147. **Selection sort** is always $O(n^2)$ comparisons but makes only $O(n)$ **swaps** and is **unstable**.
- 148. **Quick sort** is $O(n \log n)$ average but $O(n^2)$ worst (sorted input with end pivot), **in-place**, **unstable**.
- 149. Quick sort uses $O(\log n)$ stack space and avoids worst case via **randomized/median-of-three** pivots.
- 150. **Merge sort** is $O(n \log n)$ in **all 3 cases**, **stable**, but needs $O(n)$ extra space.
- 151. **Heap sort** is $O(n \log n)$ in all cases, **in-place ($O(1)$)**, but **unstable**.
- 152. **Shell sort** uses a **gap/increment** sequence; worst case is $O(n^2)$ or $O(n^{1.5})$ depending on gaps.
- 153. **Counting sort** runs in $O(n + k)$ for a small integer range **k** and is **stable**.
- 154. **Radix sort** runs in $O(d \times (n + b))$ over **d digits** with base **b**, processing **LSD-first**, and is **stable**.
- 155. **Bucket sort** averages $O(n + k)$ but worst case $O(n^2)$ when all elements fall in **one bucket**.
- 156. **Quick sort** is usually **fastest in practice** despite its $O(n^2)$ worst case.
- 157. For **linked lists**, **merge sort** is preferred (no random access needed, $O(1)$ node relinking).
- 158. **3** Divide-and-Conquer algorithms are **merge sort, quick sort and binary search**.
- 159. Selection sort makes the **fewest swaps ($O(n)$)** among simple quadratic sorts.
- 160. **Bubble and Insertion** sort reach $O(n)$ best case on already-sorted input.

Merging

- 161. Merging **2** sorted lists of size m and n takes $O(m + n)$ using **two pointers**.
- 162. A **k-way merge** of k sorted lists uses a **min-heap** in $O(N \log k)$ time.
- 163. **External merge sort** sorts in-memory **runs**, then performs a **k-way merge** from disk.

Hashing

- 164. **Hashing** gives **average $O(1)$** insert, search and delete via a **hash function**.
- 165. The **load factor $\alpha = n / m$** (items $\div$ slots) governs hash-table performance.
- 166. A good hash function is **fast, uniform** and **collision-minimising**.
- 167. There are **4** common hash functions: **division, mid-square, folding and multiplication**.
- 168. The **division method $h(k) = k \bmod m$** works best when **m is a prime** number.
- 169. The **multiplication method** uses Knuth's constant **$A \approx 0.6180339$** (golden ratio).
- 170. A **collision** is when **2** keys map to the **same** hash index.
- 171. Collision resolution is **2** broad types: **open hashing (chaining)** and **closed hashing (open addressing)**.
- 172. **Separate chaining** stores collided keys in a **linked list** per slot, search time **$O(1 + \alpha)$** .
- 173. **3** open-addressing schemes are **linear probing, quadratic probing and double hashing**.
- 174. **Linear probing** causes **primary clustering**; **quadratic probing** causes **secondary clustering**.
- 175. **Double hashing** uses **2** hash functions and minimises clustering — the **best** open-addressing scheme.
- 176. **Rehashing** rebuilds a **larger table** when the **load factor** grows too high.

PASCAL & Programming Constructs

- 177. **PASCAL**, by **Niklaus Wirth (1970)**, is a **strongly-typed, structured** language ending each program with a **period**.
- 178. PASCAL has **3** loop constructs: **for, while-do and repeat-until**.
- 179. **while-do** is **entry-controlled** (may run **0** times); **repeat-until** is **exit-controlled** (runs **at least once**).
- 180. **break** exits a loop immediately; **continue** skips to the **next iteration**.
- 181. A **function** returns a value while a **procedure** returns **none**.
- 182. Parameter passing is **2** kinds: **call by value** (copy) and **call by reference** (PASCAL **var**).
- 183. **Call by reference** lets the callee **modify** the caller's original variable; **call by value** cannot.
- 184. **Recursion** needs a **base case** to terminate and uses the **system stack** for activation records.
- 185. A PASCAL **record** groups **heterogeneous** fields, like a C **struct**.

Deep-Dive Drill — Complexity & Recursion

- 186. The **base of the logarithm** in **$O(\log n)$** is irrelevant since it differs by a **constant factor**.
- 187. Naive recursive **Fibonacci** is **$O(2^n)$** ($\approx \phi^n$, golden ratio), while iterative Fibonacci is **$O(n)$** .
- 188. **Recursive factorial** is **$O(n)$** time with **$O(n)$** stack depth.
- 189. A **recursion tree** for $T(n) = aT(n/b) + f(n)$ has **$\log_b n$** levels and **$a^{\log_b n}$** leaves.
- 190. The space complexity of recursion equals its **maximum stack depth**, often **$O(\log n)$** or **$O(n)$** .
- 191. **Tail recursion** can be optimised by compilers into a **loop**, using **$O(1)$** stack space.
- 192. A nested loop where the inner runs to the outer index sums to **$n(n-1)/2$** , still **$O(n^2)$** .
- 193. Master's Theorem **does not apply** to **$T(n) = T(n-1) + n$** (subtract-and-conquer, solving to **$O(n^2)$**).
- 194. Selection and Bubble sort have the **same $\Theta(n^2)$** comparisons but differ in **swaps**.
- 195. **$O(n \log n)$** sits strictly between **$O(n)$** and **$O(n^2)$** for large n .

Deep-Dive Drill — Arrays & Matrices

- 196. A 2-D array $A[m][n]$ occupies **$m \times n \times w$** bytes of contiguous memory.
- 197. In **row-major** the **last** index varies fastest; in **column-major** the **first** index varies fastest.
- 198. **Binary search needs an array** (random access), so it cannot run efficiently on a **linked list**.
- 199. Merging **2** sorted arrays of sizes m and n produces a sorted array in **$O(m + n)$** .
- 200. A **3-D array** $A[i][j][k]$ address adds three index terms scaled by the respective dimension products.
- 201. **Insertion at the front** of an array shifts all **n** elements, costing **$O(n)$** .
- 202. A **sparse matrix** with t non-zeros and r rows stores **$t + 1$** triplet rows (including the header).
- 203. Arrays provide **locality of reference**, making them **cache-friendly** versus linked lists.

Deep-Dive Drill — Linked Lists

- 204. Deleting a node in a **DLL** given its pointer is **O(1)**; in an **SLL** it needs the **previous** node.
- 205. To delete an SLL node given **only its pointer**, copy the **next node's data** and skip the next node.
- 206. Inserting into a **DLL** updates **2** pointers per neighbour; an SLL updates only **1**.
- 207. The **Josephus problem** is naturally modelled with a **circular linked list**.
- 208. After Floyd's pointers meet, moving one pointer to the **head** and advancing both by **1** finds the **loop start**.
- 209. A circular list traversal stops when the pointer returns to the **starting/head** node.
- 210. An **XOR linked list** stores **prev XOR next** in one field to save memory in a doubly linked list.
- 211. **Polynomial addition** via linked lists merges terms with **equal exponents** by adding coefficients.

Deep-Dive Drill — Stacks

- 212. **Postfix** is evaluated **left-to-right**, while **prefix** is evaluated **right-to-left**.
- 213. A **min-stack** supports **getMin in O(1)** using an auxiliary stack of minimums.
- 214. **Balanced parenthesis** checking pushes openers and pops on each matching **closer**.
- 215. The **system stack** stores the **return address, parameters and local variables** per call.
- 216. Converting **infix to postfix** reverses the expression, applies postfix logic, then reverses again.
- 217. **DFS** of a graph is implemented iteratively with **1** explicit stack.
- 218. Evaluating **2 3 4 * +** (postfix) yields **14** ($2 + 3 \times 4$).
- 219. An empty stack pop triggers **underflow**; a full array stack push triggers **overflow**.

Deep-Dive Drill — Queues

- 220. A circular queue either wastes **1** slot or keeps a **count** variable to tell full from empty.
- 221. A **deque** allows **O(1)** insertion and deletion at **both** ends.
- 222. A **priority queue** is **not FIFO**; order depends on **priority**, with ties broken by arrival.
- 223. **Round-robin** CPU scheduling uses a **circular queue** of ready processes.
- 224. The **front** pointer advances on **dequeue**; the **rear** pointer advances on **enqueue**.
- 225. A **double-ended priority queue** supports min and max extraction; built on a **min-max heap**.
- 226. A printer **spool** is a real-world **FIFO queue** of pending jobs.

Deep-Dive Drill — Trees

- 227. The **degree** of a node is its number of **children**; a binary tree has max node degree **2**.
- 228. A binary tree of **n** nodes stored in an array (1-based) puts node **i**'s children at **2i** and **2i+1**.
- 229. A **skewed** binary tree of **n** nodes has height **n - 1** and behaves like a **linked list**.
- 230. A **complete** binary tree of **n** nodes has height **$\lceil \log_2 n \rceil$** .
- 231. The number of **leaf nodes** in a full binary tree with **i** internal nodes is **i + 1**.
- 232. **Inorder + Preorder** uniquely rebuild a tree; **Preorder + Postorder alone cannot** (except full trees).
- 233. **Postorder** is used to **free/delete** a tree; **preorder** is used to **copy/serialize** it.
- 234. A **BST** built from **sorted** input degenerates into a **skewed** tree with **O(n)** operations.
- 235. The **inorder successor** of a BST node is the **leftmost** node of its **right subtree**.
- 236. A **threaded binary tree** allows inorder traversal in **O(n)** with **O(1)** extra space (no stack).
- 237. **AVL** trees are **more rigidly balanced** than **Red-Black** trees, giving faster lookups but costlier inserts.
- 238. A **Red-Black tree** allows at most **2x** the shortest path as the longest root-to-leaf path.
- 239. **B-trees** minimise **disk I/O** by storing **many keys per node**, matching a disk **block**.
- 240. A **B+ tree's leaf nodes** form a **linked list**, enabling efficient **sequential/range** scans.
- 241. The **height of an AVL tree** with **n** nodes is **O(log n)**, bounded by about **1.44 log₂(n+2)**.
- 242. An **expression tree's postorder** traversal directly yields the **postfix** expression.
- 243. **OBST** places **high-frequency keys nearer the root** to cut the **expected search cost**.
- 244. A **min-heap** is a complete binary tree, but a **BST** need not be complete.

245. The **maximum number of nodes** in a binary tree of height h (edges) is $2^{h+1} - 1$.

Deep-Dive Drill — Graphs

246. A **tree** is a **connected acyclic** graph with exactly $n - 1$ edges.
247. A graph with n vertices has at most $n(n-1)/2$ undirected edges (simple graph).
248. **In-degree + out-degree** of a vertex in a digraph equals its **total degree**.
249. **DFS** classifies edges into **tree, back, forward and cross** edges.
250. A **back edge** in DFS indicates a **cycle** in the graph.
251. **BFS from a source** computes the **minimum number of edges** to each vertex (unweighted shortest path).
252. **Kruskal's** uses **Union-Find** to reject edges that would form a **cycle**.
253. **Prim's** always maintains **1** connected tree; **Kruskal's** may maintain a **forest** until the end.
254. **Dijkstra** greedily finalises the vertex with the **smallest tentative distance** each step.
255. **Bellman-Ford** relaxes all edges $V - 1$ times; a further relaxation signals a **negative cycle**.
256. **Floyd-Warshall** uses each vertex as an **intermediate** in its triple-nested $O(V^3)$ loop.
257. **Topological sort** orders a DAG so every edge $u \rightarrow v$ has u **before** v .
258. **Kahn's algorithm** repeatedly removes vertices with **in-degree 0** for topological sort.
259. A **strongly connected component** is found via **Tarjan's** or **Kosaraju's** algorithm in $O(V+E)$.
260. The **adjacency list** total size equals $V + 2E$ for an undirected graph (each edge counted twice).
261. A **complete graph** K_n is always **connected** and has **degree $n - 1$** at every vertex.
262. **Dijkstra with a Fibonacci heap** achieves $O(E + V \log V)$, the best known bound.

Deep-Dive Drill — Sorting & Searching

263. **Bubble sort** performs at most $n(n-1)/2$ comparisons in the worst case.
264. **Insertion sort** is an **online** algorithm — it can sort data as it **arrives**.
265. **Selection sort** is **not adaptive** — it runs $O(n^2)$ even on a sorted array.
266. **Quick sort's** worst case $O(n^2)$ happens when the pivot is always the **smallest or largest** element.
267. The **partition** step of quick sort is $O(n)$ and places the pivot in its **final position**.
268. **Merge sort** is the preferred algorithm for **external sorting** of huge files.
269. **Heap sort** first builds a heap in $O(n)$ then does n deletions of $O(\log n)$ each.
270. **Counting sort** is efficient only when the key range $k = O(n)$.
271. **Radix sort** must use a **stable** intermediate sort (usually counting) to be correct.
272. **Shell sort** improves insertion sort by allowing exchanges of **far-apart** elements first.
273. **Bucket sort** assumes inputs are **uniformly distributed** over a range.
274. The only sorts that beat $\Omega(n \log n)$ are **non-comparison** sorts: **counting, radix, bucket**.
275. **Adaptive** sorts (Bubble-optimised, Insertion) speed up on **nearly sorted** input.
276. For **small n** , **insertion sort** often outperforms **quick/merge** due to low overhead.
277. **Stability matters** when sorting records by a **secondary key** after a primary key.
278. **Binary search** makes at most $\lceil \log n \rceil + 1$ comparisons.
279. **Interpolation search** estimates position using the **value** of the key, not just the midpoint.
280. **Ternary search** divides into **3** parts but does **more comparisons** than binary search.
281. The **decision tree** for n elements has at least $n!$ leaves, giving the $\log(n!) = \Omega(n \log n)$ bound.
282. **Quick sort is unstable** because partitioning can **swap equal keys** across the pivot.

Deep-Dive Drill — Hashing

283. With **separate chaining**, the average search cost is $O(1 + \alpha)$ where α is the load factor.
284. In **open addressing**, the load factor α **must stay < 1** (table cannot overflow its slots).
285. **Linear probing** with step 1 examines slots $h, h+1, h+2, \dots$ modulo m .
286. **Quadratic probing** examines slots $h, h+1, h+4, h+9, \dots$ modulo m .

- 287. **Double hashing** uses step size $h(k)$, which must be **relatively prime** to the table size m .
- 288. **Primary clustering** forms long runs of filled slots, worsening **linear probing**.
- 289. A **perfect hash function** maps a static key set with **zero collisions**.
- 290. **Rehashing** typically **doubles** the table size and keeps the load factor near **0.5–0.75**.
- 291. The **division method** avoids table sizes that are **powers of 2** to use all key bits.
- 292. **Hashing** offers **$O(1)$ average** but **$O(n)$ worst-case** search when all keys collide.

Rapid-Fire Mixed Exam Pointers

- 293. **Greedy** algorithms include **Prim, Kruskal, Dijkstra, Huffman and fractional knapsack**.
- 294. **Dynamic programming** algorithms include **Floyd-Warshall, Bellman-Ford, OBST, LCS and matrix-chain**.
- 295. **Divide-and-conquer** algorithms include **merge sort, quick sort, binary search and Strassen's**.
- 296. **Backtracking** solves **N-Queens, Sudoku, graph colouring and the maze** problem.
- 297. The **Tower of Hanoi** for 3 disks needs **7** moves ($2^3 - 1$).
- 298. **DFS and BFS** both run in **$O(V + E)$** on an adjacency list.
- 299. **Prim's and Dijkstra's** are structurally similar **greedy** algorithms using a **priority queue**.
- 300. A **complete binary tree** is the shape used by both a **heap** and an **array-based** tree.
- 301. **AVL, Red-Black, B and B+ trees** are all **self-balancing** search trees with **$O(\log n)$** operations.
- 302. **Stack** enables **DFS, recursion and expression evaluation**; **Queue** enables **BFS and scheduling**.
- 303. The **fastest average sort** in practice is **quick sort**; the **guaranteed $O(n \log n)$ stable** sort is **merge sort**.
- 304. **Hashing** gives the fastest average **search $O(1)$** , beating BST's **$O(\log n)$** .
- 305. **Negative-weight** single-source shortest paths require **Bellman-Ford**, not **Dijkstra**.
- 306. **All-pairs shortest path** via **Floyd-Warshall** is **$O(V^3)$** time and **$O(V^2)$** space.
- 307. A **binary heap** supports **find-min/max in $O(1)$** and **insert/delete in $O(\log n)$** .
- 308. **Catalan(n)** counts both **binary tree shapes** and **valid parenthesis** sequences with n pairs.

Atomic Round 2 — Trees & Graph Theory

- 309. A **full binary tree** always has an **odd** number of nodes (since $n_{\text{leaf}} = n_{\text{int}} + 1$, no degree-1 nodes).
- 310. The **minimum nodes** in an AVL tree of height h follow **$N(h) = N(h-1) + N(h-2) + 1$** (Fibonacci-like).
- 311. A **2-3-4 tree** is a **B-tree of order 4** and is isometric to a **Red-Black tree**.
- 312. In an extended binary tree with n internal nodes, **$EPL = IPL + 2n$** (external vs internal path length).
- 313. **Cayley's formula** says the complete graph **K_n** has **n^{n-2}** distinct spanning trees.
- 314. An **Euler circuit** exists iff the graph is connected and **every vertex has even degree**.
- 315. An **Euler path** exists iff exactly **0 or 2** vertices have **odd degree**.
- 316. A graph is **bipartite** iff it is **2-colourable** iff it has **no odd-length cycle**.
- 317. **Euler's formula** for a connected planar graph is **$V - E + F = 2$** .
- 318. A simple connected **planar** graph satisfies **$E \leq 3V - 6$** for **$V \geq 3$** .
- 319. **Strassen's** matrix multiplication runs in about **$O(n^{2.81}) = O(n^{\log_2 7})$** .
- 320. A **Hamiltonian path** visits every **vertex** once; an **Euler path** visits every **edge** once.
- 321. A **2-3 tree** is a **B-tree of order 3** with all leaves at the same level.
- 322. The number of **NULL pointers** in a binary tree of n nodes is **$n + 1$** .
- 323. A **min-max heap** supports both **$O(1)$ min and max** with **$O(\log n)$ updates**.
- 324. **Kruskal's** processes edges in **non-decreasing** weight order; **Prim's** by nearest vertex.
- 325. A **disconnected** graph of n vertices needs at least **$n - 1$** edges to become a spanning **tree**.

Atomic Round 2 — Searching, Sorting & Hashing

- 326. **Jump search** on a sorted array runs in **$O(\sqrt{n})$** by jumping fixed blocks.
- 327. **Exponential search** finds a range in **$O(\log n)$** then binary-searches it.
- 328. **Insertion sort** runs in **$O(n + d)$** time where **d** is the number of **inversions**.

- 329. **Bubble sort** needs at most **$n - 1$ passes** to fully sort n elements.
- 330. **Selection sort** always performs exactly **$n - 1$ passes** regardless of input order.
- 331. **Cuckoo hashing** uses **2** hash tables to guarantee **$O(1)$ worst-case** lookup.
- 332. The **birthday paradox** explains why hash **collisions** appear sooner than intuition suggests.
- 333. **Universal hashing** picks a hash function **randomly** to bound expected collisions.
- 334. **Ternary search** makes about **$2 \log n$** comparisons, more than binary search's **$\log n$** .
- 335. **Merge sort** performs exactly **$\log n$** levels of merging.
- 336. **Pigeonhole / counting** sorts are stable and run in **$O(n + \text{range})$** .
- 337. A **stable** sort is required when sorting by **multiple keys** sequentially.
- 338. **Timsort** (Python/Java) is a hybrid of **merge and insertion** sort, stable, **$O(n \log n)$** .
- 339. **Introsort** (C++ std::sort) combines **quick, heap and insertion** sort to avoid **$O(n^2)$** .
- 340. The **median-of-medians** algorithm finds the k -th smallest element in **$O(n)$** worst case.

Atomic Round 2 — Operations, Conditions & Values

- 341. Array stack: **overflow at $\text{TOP} = n - 1$, underflow at $\text{TOP} = -1$** .
- 342. Circular queue: **full when $(\text{rear}+1)\%n == \text{front}$, empty when $\text{front} == -1$** .
- 343. A heap of n nodes has **$n/2$** leaf nodes and **$n/2$** internal nodes.
- 344. The **last internal node** of a heap of n elements is at index **$n/2$** (1-based).
- 345. **Build-heap** sifts down from index **$n/2$** to **1**, costing **$O(n)$** .
- 346. A binary tree level l holds between **1** and **2^l** nodes.
- 347. A graph traversal visits each vertex **once** and each edge **at most twice** (undirected).
- 348. **DFS recursion depth** can reach **$O(V)$** , risking stack overflow on deep graphs.
- 349. **Dijkstra** stores tentative distances initialised to **infinity** except the source at **0**.
- 350. **Bellman-Ford** initialises distances to **infinity** and relaxes edges **$V - 1$** times.
- 351. A **spanning tree** has **no cycles** and connects all **V** vertices with **$V - 1$** edges.
- 352. **Postfix evaluation** of n tokens uses a stack of max depth about **$n/2$** .
- 353. **Quick sort** recursion depth is **$O(\log n)$** best/average and **$O(n)$** worst.
- 354. A **perfect** binary tree of height h has exactly **$2^{h+1} - 1$** nodes and **2^h** leaves.
- 355. The number of **comparisons** in selection sort is always **$n(n-1)/2$** .

Final Rapid-Fire — Discriminators

- 356. **Array vs List**: array = **$O(1)$ access / $O(n)$ insert**; list = **$O(n)$ access / $O(1)$ insert**.
- 357. **Stack vs Queue**: stack = **LIFO (one end)**; queue = **FIFO (two ends)**.
- 358. **BFS vs DFS**: BFS = **queue, shortest unweighted path**; DFS = **stack, cycle/topo-sort**.
- 359. **Prim vs Kruskal**: Prim = **vertex-based, dense**; Kruskal = **edge-based + union-find, sparse**.
- 360. **Dijkstra vs Bellman-Ford**: Dijkstra = **greedy, no negatives**; Bellman-Ford = **DP, negatives + cycle detection**.
- 361. **B-tree vs B+ tree**: B-tree = **data in all nodes**; B+ tree = **data only in linked leaves**.
- 362. **Linear vs Quadratic probing**: linear = **primary clustering**; quadratic = **secondary clustering**.
- 363. **Merge vs Quick sort**: merge = **stable, $O(n)$ space, always $n \log n$** ; quick = **in-place, unstable, $O(n^2)$ worst**.
- 364. **Big-O vs Big-Theta**: O = **upper bound only**; Θ = **upper and lower (tight)**.
- 365. **Complete vs Full tree**: complete = **last level left-filled**; full = **every node has 0 or 2 children**.
- 366. **AVL vs Red-Black**: AVL = **stricter balance, faster search**; RB = **looser, faster insert/delete**.
- 367. **Inorder vs Preorder**: inorder of BST = **sorted**; preorder first node = **root**.
- 368. **Internal vs External sort**: internal = **fits in RAM**; external = **uses disk + merge**.
- 369. **Min-heap vs Max-heap**: min-heap root = **smallest**; max-heap root = **largest**.
- 370. **Greedy vs DP**: greedy = **never reconsiders**; DP = **reuses overlapping subproblems**.

Atomic Round 3 — Advanced Structures & Algorithms

371. A **Trie (prefix tree)** searches/inserts a key of length L in $O(L)$, independent of the number of keys.
372. **Tries** power **autocomplete, spell-check and IP routing** via shared prefixes.
373. A **segment tree** answers **range queries in $O(\log n)$** with $O(n)$ build and $O(n)$ space.
374. A **Fenwick tree (Binary Indexed Tree)** does **prefix-sum update and query in $O(\log n)$** .
375. A **skip list** gives **expected $O(\log n)$** search/insert using **randomised** multi-level links.
376. A **prefix-sum array** answers any **range-sum in $O(1)$** after $O(n)$ preprocessing.
377. **Kadane's algorithm** finds the **maximum subarray sum in $O(n)$** .
378. The **Dutch National Flag** algorithm 3-way partitions an array in $O(n)$ (used in 3-way quicksort).
379. **Huffman coding** is a **greedy $O(n \log n)$** method producing **optimal prefix-free** codes.
380. A **Huffman tree** assigns **shorter codes to more frequent** symbols, minimising total bits.
381. A **Bloom filter** is a **probabilistic** set with possible **false positives** but **no false negatives**.
382. **Consistent hashing** minimises key remapping when nodes are added/removed in **distributed caches**.
383. The **KMP** string-matching algorithm runs in $O(n + m)$ using a **prefix (failure) function**.
384. **Rabin-Karp** matches strings using **rolling hashes in average $O(n + m)$** , worst $O(nm)$.
385. **Ford-Fulkerson** computes **maximum flow**, equal to the **minimum cut** (max-flow min-cut theorem).
386. A^* search is **optimal** when its heuristic is **admissible** (never overestimates).
387. **Hoare's partition** scheme does **fewer swaps** than **Lomuto's** in quick sort.
388. **Union-Find** with path compression + union by rank runs in near $O(\log^* n)$ (inverse Ackermann).
389. An **LRU cache** achieves $O(1)$ get/put using a **hash map + doubly linked list**.
390. A **suffix array** supports substring search in $O(m \log n)$ with $O(n)$ space.
391. A **deque-based** sliding-window maximum runs in $O(n)$ total (each element pushed/popped once).
392. **Matrix-chain multiplication** uses **DP** to find the optimal parenthesization in $O(n^3)$.
393. The **Longest Common Subsequence (LCS)** is solved by **DP** in $O(m \times n)$.
394. The **0/1 Knapsack** is solved by **DP** in $O(nW)$ (pseudo-polynomial), or **Branch-and-Bound**.
395. **Counting inversions** in an array is done with a modified **merge sort** in $O(n \log n)$.
396. A **monotonic stack** finds the **next greater element** for all items in $O(n)$.
397. **Reservoir sampling** picks k random items from a stream of unknown size in $O(n)$ with $O(k)$ space.
398. **Topological sort + DP** gives **single-source shortest paths in a DAG** in $O(V + E)$.
399. The **Boyer-Moore voting** algorithm finds a **majority element** in $O(n)$ time, $O(1)$ space.
400. **Quickselect** finds the k -th smallest element in **average $O(n)$** , worst $O(n^2)$.
401. A **balanced BST (AVL/RB)** supports **ordered** operations (rank, range) that a **hash table cannot**.
402. **Memoization** is **top-down DP**; **tabulation** is **bottom-up DP** — both eliminate recomputation.
403. The **knapsack-style** problems show the **time-space trade-off** of DP tables.
404. A **graph with V vertices and V edges** must contain **exactly one cycle** (if connected).
405. **Disjoint-set forests** answer **connectivity queries** incrementally, unlike static DFS.
406. **Heapify (sift-down)** of a single node costs $O(\log n)$, but building the whole heap costs $O(n)$.
407. The **height of a complete binary tree** with n nodes is $\lceil \log_2(n + 1) \rceil - 1$.
408. A **stable, in-place, $O(n \log n)$** comparison sort is **theoretically hard**; merge is stable but not in-place, heap is in-place but unstable.